Next-Generation Malware Detection: Fusion of Static and Dynamic Analysis for Family Classification

SUBMITTED BY

Aparna Pradeep Nair

Seat No.

DATE OF SUBMISSION

Under the guidance of

Dr. Anupama Jawale

Submitted in partial fulfilment of the requirements for qualifying M.Sc. Part II Examination

UNIVERSITY OF MUMBAI



UNIVERSITY DEPARTMENT OF INFORMATION TECHNOLOGY
IT PARK, 2ND FLOOR, UNIVERSITY DEPARTMENT OF INFORMATION TECHNOLOGY, UNIVERSITY OF MUMBAI , KALINA, SANTACRUZ EAST MUMBAI- 400098

University of Mumbai



Next-Generation Malware Detection: Fusion of Static and Dynamic Analysis for Family Classification

SUBMITTED BY

Aparna Pradeep Nair

Seat No.

DATE OF SUBMISSION

Under the guidance of

Dr. Anupama Jawale

Submitted in partial fulfilment of the requirements for qualifying M.Sc. Part II Examination

Department of Information Technology

It Park, 2nd Floor, University Department of Information Technology,
University of Mumbai , Kalina, Santacruz east Mumbai- 400098

University of Mumbai



Department of Information Technology

PROJECT CERTIFICATE

This is to certify that the Project entitled _____ submitted by

Mr./Ms. _____

(Seat No.) in partial fulfilment for M.Sc. Degree Examination has been found satisfactory.

This report had not been submitted for any other examination and does not Form part of

any other course undergone by the candidate.

It is further certified that he/she has completed the project using research methodology.

Signature	Signature	Signature
Lecturer Incharge Guided by	External Examiner Examined By	Head of the Department Certified by

College Stamp

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who have supported and guided us throughout the successful completion of this research project.

First and foremost, we are deeply grateful to our project guide, **Dr. Anupama Jawale**, for his constant guidance, valuable suggestions, and encouragement. His expertise and insightful feedback played a crucial role in shaping this research work on Next-Generation Malware Detection.

We extend our heartfelt thanks to the **Head of the Department** and all the faculty members of the Department of Information Technology, University of Mumbai, for providing us with the necessary academic environment and support during the course of this project.

We are also thankful to the University of Mumbai for giving us the opportunity to undertake this research as part of the M.Sc. Part II curriculum.

Finally, we would like to express our appreciation to our families and friends for their continuous support, patience, and motivation throughout this journey.

Without the collective support of all the above, this project would not have been possible.

-Aparna Pradeep Nair

INDEX

SR.NO	Description	Page Number
1	Chapter 1: Introduction	
2	Chapter 2: Literature Survey	
3	Chapter 3: Methodology / Approach	
4	Chapter 4: Proposed Design / UI design	
5	Bibliography - references and links.	

CHAPTER 1

INTRODUCTION

1.1 Background

Malicious software, or malware, is any software that performs actions that harm a user, a computer or a network. It includes viruses, trojan horses, worms, rootkits, ransomware, scareware and spyware. Malware plays a central role in most computer intrusions and security incidents, and it targets individuals, enterprises and critical national infrastructure alike. With millions of malicious programs in circulation and new variants appearing daily, malware analysis has become a core part of modern incident response. Malware analysis is the practice of dissecting a malicious program to understand how it works, how to identify it, and how to defeat or eliminate it.

1.2 Evolution of Malware

Malware has evolved considerably since the 1970s, when experimental self-replicating programs such as Creeper appeared. In the 1980s, floppy-disk-borne viruses such as Brain spread between machines. Macro viruses and mass-mailing worms such as Melissa and ILOVEYOU followed in the 1990s and early 2000s. In recent years, malware has become far more sophisticated and financially or politically motivated. Modern threats include ransomware, spyware and information stealers, advanced persistent threats (APTs) and Malware-as-a-Service (MaaS) ecosystems. These threats use cross-platform capabilities, artificial intelligence and advanced evasion techniques such as obfuscation, packing, polymorphism and sandbox detection.

1.3 Malware Detection and Analysis

The primary objective of malware analysis is to determine the capabilities of a suspected binary, identify its behaviour on a system or network, and assess the impact of an infection. Two broad families of techniques are used:

- **Static analysis** inspects a sample without executing it, extracting structural and code-level attributes such as Portable Executable (PE) headers, imports, sections and entropy. It is fast and scalable but is weakened by packing, encryption and obfuscation.
- **Dynamic analysis** executes the sample in a controlled, isolated environment and observes runtime behaviour such as processes, network connections and file activity. It reveals hidden behaviour but is costly and can be evaded by sandbox-aware malware.

Because each technique has complementary strengths and weaknesses, combining them is an attractive direction.

1.4 Problem Statement

Existing malware detection systems frequently depend on a single analysis technique and may have difficulty detecting obfuscated, packed, polymorphic or previously unseen malware. Static-only systems can be defeated by packing and obfuscation, while dynamic-only systems can be bypassed by malware that detects the analysis environment and stays

dormant. Few systems provide an integrated, automated pipeline that fuses both kinds of evidence and also classifies the sample into a malware family. This leads to real-world problems:

1. **Zero-day attacks:** new variants bypass signature-based antivirus.
2. **SOC overload:** high false-positive rates waste analyst time and cause alert fatigue.
3. **Slow incident response:** without family classification, responders cannot easily predict attacker behaviour or choose countermeasures.
4. **Threat-intelligence blind spots:** incomplete detection makes it harder to link attacks to known families such as WannaCry, Zeus or Emotet.

1.5 Research Motivation

The motivation of this work is to develop an integrated system that combines the complementary static and dynamic characteristics of a file within a single machine learning pipeline. By bridging academic research and practical industry needs, the project aims to contribute to the development of next-generation Endpoint Detection and Response (EDR) capabilities. A basic understanding of malware is also valuable for non-technical users, who are frequently targeted through phishing, ransomware and data-theft campaigns, so accessible and interpretable tooling has wider societal value.

1.6 Research Objectives

- To develop an automated malware analysis pipeline.
- To extract meaningful static features from executable files.
- To observe runtime behaviour through controlled dynamic analysis.
- To combine static and dynamic features through feature fusion.
- To apply machine-learning-based malware detection.
- To support malware family classification.
- To provide interpretable results through confidence scores, feature contributions and reporting.
- To develop the solution as a practical desktop-based application.

1.7 Scope

The system focuses on offline malware analysis, Windows PE-oriented static analysis, controlled dynamic monitoring and machine-learning-based classification. It does **not** include real-time detection or kernel-level monitoring, which are identified as future work. The effectiveness of the system depends on the quality and diversity of the training data, and dynamic analysis is limited by how well a sandbox can replicate real execution conditions.

1.8 Significance

The project provides a more comprehensive view of a suspicious file by combining structural and behavioural information. It can help security teams reduce false positives, detect obfuscated or unseen variants more reliably, identify malware families faster, and understand why a file was classified as it was. The modular, desktop-based design also makes it suitable for education and practical analysis.

CHAPTER 2

LITERATURE SURVEY

Malware detection uses specialised methods and tools to spot, block and neutralise malicious software before it compromises systems, networks or devices. Traditional antivirus relied on signature matching, but the growing complexity of malware, together with a global shortage of skilled security professionals, has pushed organisations toward more automated, multi-layered strategies.

2.1 Signature-Based Detection

Signature-based systems identify malware by matching byte sequences, hashes or code patterns against a database of known threats. They have low computational overhead and high accuracy on known samples, but they are reactive. Minor changes such as polymorphism or packing can make a signature obsolete, so they struggle with zero-day and evolving malware. This limitation motivated the move toward behaviour-based and learning-driven approaches.

2.2 Static Analysis

Static analysis examines a program without executing it. Common features include PE header attributes, imported DLLs and functions, resources, sections, opcode sequences, strings and byte-level statistics such as entropy. Studies combining such features with machine learning have shown that benign and malicious software can be separated at scale. Static analysis is efficient and suitable for early screening, but packing, encryption and obfuscation conceal or distort the features it depends on, so it is insufficient on its own. In this project the static module extracts PE header information, section entropy and suspicious API usage.

2.3 Dynamic Analysis

Dynamic analysis executes a sample in an isolated environment such as a virtual machine or sandbox and records behaviour such as system and API calls, process creation, file system changes, registry changes and network communication. Because it observes actual behaviour, it is less affected by packing and obfuscation. Its drawbacks are computational cost, limited scalability, and sandbox-aware malware that suppresses malicious behaviour when it detects an analysis environment. Hypervisor-assisted approaches have been proposed to make monitoring harder to detect. In this project the dynamic module monitors new processes, network connections and dropped files.

2.4 Machine Learning-Based Detection

Machine learning overcomes the rigidity of rule- and signature-based methods by learning patterns from labelled data. Surveys cover a wide range of techniques. Support Vector Machines, Random Forests and gradient boosting are widely used because they handle high-dimensional structured data well. Deep learning approaches learn representations directly from raw bytes, executable images , or behavioural and API-call sequences. Behaviour-based classification and clustering have also been applied to malware families. Remaining challenges include data quality, class imbalance, interpretability and generalisation to unseen variants.

2.5 Hybrid Static and Dynamic Analysis

Hybrid approaches combine structural features with runtime behaviour. Comparative work reports that hybrid methods improve accuracy and robustness over either approach alone and neural architectures that integrate both feature types have been applied to family classification. Hybrid systems therefore resist both static-evasion (packing) and dynamic-evasion (sandbox detection) techniques. However, many existing hybrids face feature-fusion complexity, higher overhead, limited attention to family classification, and a lack of end-to-end automation.

2.6 Malware Family Classification

Identifying the family of a sample (for example trojan, ransomware, spyware or worm) helps analysts predict attacker behaviour and choose containment steps. Many studies concentrate only on binary detection (benign versus malicious), whereas family attribution requires richer and more discriminative features, which supports using fused static and dynamic evidence.

2.7 Research Gaps Identified

- Many approaches provide detection but limited or no family classification.
- A large share of systems rely on static **or** dynamic analysis in isolation, reducing robustness against evasion.
- Hybrid systems often lack a unified, automated pipeline that integrates acquisition, feature extraction, fusion, classification and reporting.
- Dataset diversity, generalisation to zero-day malware, false-positive rates and model interpretability remain open problems.

This project addresses these gaps with an integrated, automated and interpretable hybrid pipeline.

2.8 Existing Systems and Their Limitations

Existing approach	Strength	Limitation
Signature-based antivirus	Fast, low false positives on known malware	Reactive; fails on zero-day, polymorphic or modified samples
Static-only detection	Efficient, scalable, safe (no execution)	Vulnerable to packing, encryption, obfuscation, polymorphism
Dynamic / sandbox-only detection	Reveals runtime behaviour and hidden payloads	High overhead, poor scalability, sandbox-aware malware evades it
Existing ML-based systems	Learn complex patterns, adapt to new data	Depend on dataset quality and feature choice; weak interpretability and generalisation

A further limitation is that many systems operate as isolated detectors. They do not provide a complete automated pipeline covering acquisition, analysis, feature fusion, classification and reporting, which contributes to false positives, limited family classification and higher analyst workload.

CHAPTER 3

METHODOLOGY / APPROACH

3.1 Overview of the Approach

The research proposes MalXray, an integrated hybrid malware detection and family classification framework that combines static analysis, dynamic behavioural analysis and machine learning in one automated, desktop-based platform. The central idea is to fuse complementary static and dynamic features into a single feature vector so that each analysis type compensates for the weaknesses of the other.

3.2 System Architecture

The system is a modular pipeline in which each stage processes data and passes it to the next:

Input Sample → File Information → Static Analysis → Dynamic Analysis → Feature Fusion → ML Classification → VirusTotal Cross-Check → Risk Assessment → PDF Report

The principal modules are: (1) Sample Acquisition, (2) Static Analysis, (3) Dynamic Behaviour Analysis, (4) Feature Fusion and Preprocessing, (5) Machine Learning Classification, and (6) Result Visualization and Reporting. [INSERT Figure 1: system architecture diagram from your research paper.]

3.3 Sample Acquisition

The user supplies a file as the input sample. The system records its location and basic characteristics before any analysis begins.

3.4 File Information Extraction

The module extracts the file type, file size, MD5 hash and SHA-256 hash, plus archive information and details of extracted files where applicable. Hashes serve as unique identifiers and are used for the VirusTotal lookup. Archives are processed where extraction is possible.

3.5 Static Analysis

Static analysis is performed without executing the sample. The implemented module extracts PE-oriented features:

Category	Features
Identification	PE identification, file entropy
Structure	Number of sections, code size, header size, image size, entry point, timestamp, resource count
Imports	Imported DLL count, imported function count, suspicious API indicators
Metadata	Debug information, digital-signature indicator
Per-section	Section entropy

High entropy often indicates packing or encryption, and certain imported APIs (for example those used for process injection or network download) are common in malware. The extractor intentionally returns a **consistent feature schema even for non-PE files**, so the downstream pipeline never fails on unexpected input. Libraries considered or used for this stage include pefile and LIEF for PE parsing and YARA for rule-based baseline comparison.

3.6 Dynamic Behaviour Analysis

Dynamic analysis observes runtime changes in an isolated environment. The current behaviour-monitoring implementation captures newly created processes, network connections and dropped files, and flags suspicious process patterns, suspicious network ports and suspicious dropped executables or scripts. Cuckoo Sandbox and Sysmon logs were identified as tools for richer behavioural data such as API calls and registry changes.

Safety design: the execution functionality is deliberately protected so that malware is never executed automatically. Runtime detonation requires an isolated environment and explicit user confirmation.

3.7 Feature Fusion and Preprocessing

Static and dynamic features are converted into a fixed-order numerical feature vector. The implementation uses simple, interpretable concatenation-based fusion instead of an opaque neural embedding, which keeps every feature traceable to its source. The dynamic feature schema is:

- Number of new processes
- Number of new network connections
- Number of dropped files
- Suspicious process flags
- Suspicious connection flags
- Suspicious file flags
- Dynamic-data availability indicator

The pipeline log confirms that a **44-feature fused vector** was generated successfully for a test sample.

3.8 Machine Learning Classification

XGBoost is the core classifier because it handles large, high-dimensional, structured data efficiently and has proven effective in malware classification. Scikit-learn is used for baselines and preprocessing, and LightGBM and deep learning frameworks (PyTorch/TensorFlow) were identified for comparison experiments. Two modes are supported:

- **Binary classification:** Benign or Malicious.
- **Multi-class (family) classification:** Benign, Trojan, Ransomware, Spyware, Worm.

When runtime information is unavailable, the classifier falls back to **static-only mode**, and the report states this explicitly.

3.9 Explainability

For each prediction the system reports feature contributions computed with XGBoost prediction contributions. This shows which features pushed the decision toward benign or malicious, helping analysts trust and verify the verdict and addressing the interpretability gap found in the literature.

3.10 VirusTotal Cross-Check

VirusTotal is used only as a **secondary, independent validation signal**, not as the primary decision-maker. The machine learning model produces the principal verdict, and agreement or disagreement with VirusTotal supports risk assessment.

3.11 Risk Assessment and Report Generation

The ML verdict, confidence, behavioural indicators and VirusTotal result are combined into a risk assessment. The system then produces a PDF report containing file information, ML verdict, classification mode (fused or static-only), confidence, malware family, behavioural observations, VirusTotal information, feature contributions, and warnings and limitations. The report explicitly identifies cases where dynamic analysis was not performed.

3.12 Tools and Technologies

Purpose	Tools
Language	Python 3.x
ML	scikit-learn, XGBoost, LightGBM, PyTorch / TensorFlow
Static analysis	pefile, LIEF, YARA
Dynamic analysis	Cuckoo Sandbox, Sysmon logs, process / network / file monitoring
Data handling and visualisation	pandas, numpy, matplotlib, seaborn
Dataset	EMBER (static features) and sandbox-collected behavioural data

3.13 Dataset and Evaluation Plan

Static features come from datasets such as EMBER and dynamic features from sandbox execution. Models are compared in three configurations: static-only, dynamic-only and fused. Metrics are accuracy, precision, recall, F1-score, confusion matrices, false-positive rate and feature importance. [STATE your actual dataset name, number of benign and malicious samples, family distribution and train/test split.]

CHAPTER 4

PROPOSED DESIGN / UI DESIGN

4.1 Design Overview

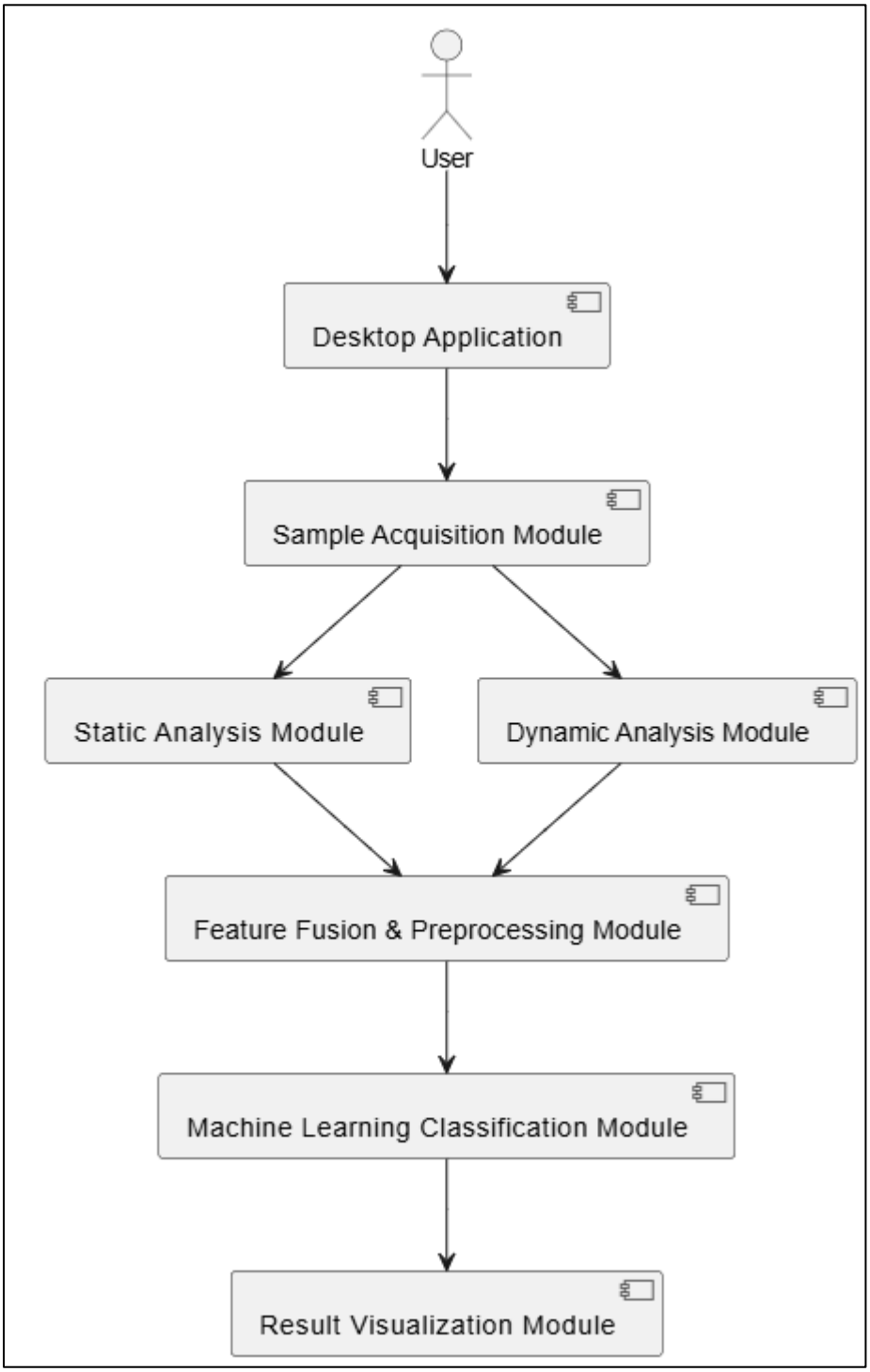
The design is a modular pipeline wrapped in a desktop application. A user selects a file, the application runs the analysis stages in order, and the result is shown on screen and exported as a PDF. Modules are loosely coupled, so each can be improved or replaced (for example swapping the classifier) without changing the others.

4.2 Module Design

Module	Input	Output
Sample Acquisition	User-selected file	File path and basic characteristics
File Information	Sample file	File type, size, MD5, SHA-256, archive contents
Static Analysis	Sample file	PE features, entropy, suspicious API indicators
Dynamic Analysis	Sample (isolated environment, user confirmation)	New processes, connections, dropped files, suspicious flags
Feature Fusion	Static and dynamic features	Fixed-order numeric vector (44 features in the tested sample)
ML Classification	Fused vector	Verdict, family, confidence, feature contributions
VirusTotal Check	File hash	Independent detection result
Risk Assessment and Reporting	All previous outputs	On-screen result and PDF report

4.3 Data Flow and Fallback Logic

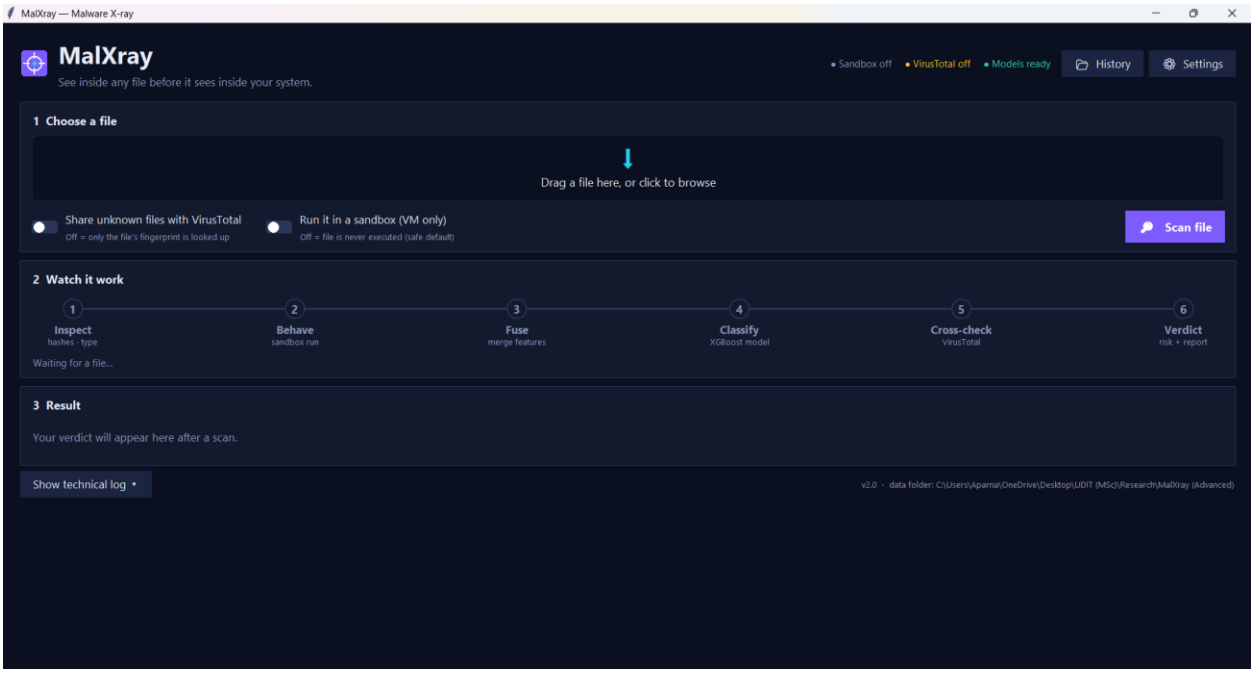
The sample passes through file-information extraction and static analysis. If a safe isolated environment is available and the user confirms, dynamic analysis runs and its features are fused with static ones. If dynamic data is unavailable, the dynamic availability indicator is set accordingly and the static-only fallback model is used.



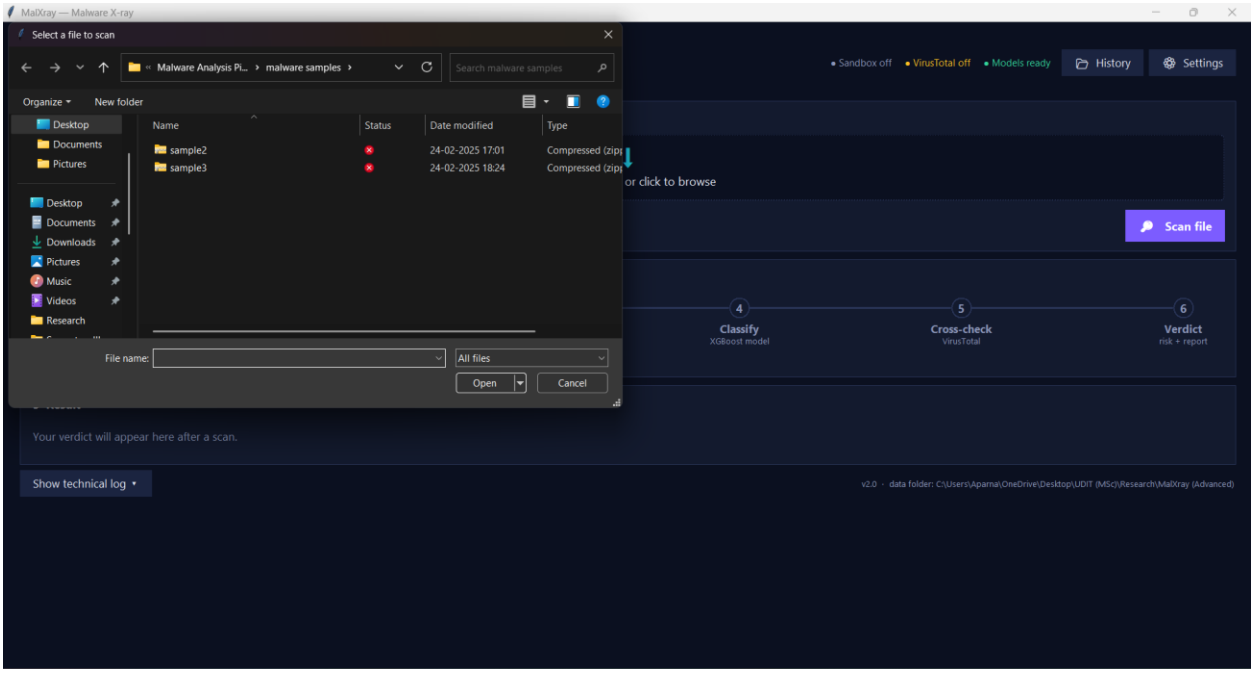
4.4 User Interface Design

The desktop interface is designed to be simple and analyst-friendly.

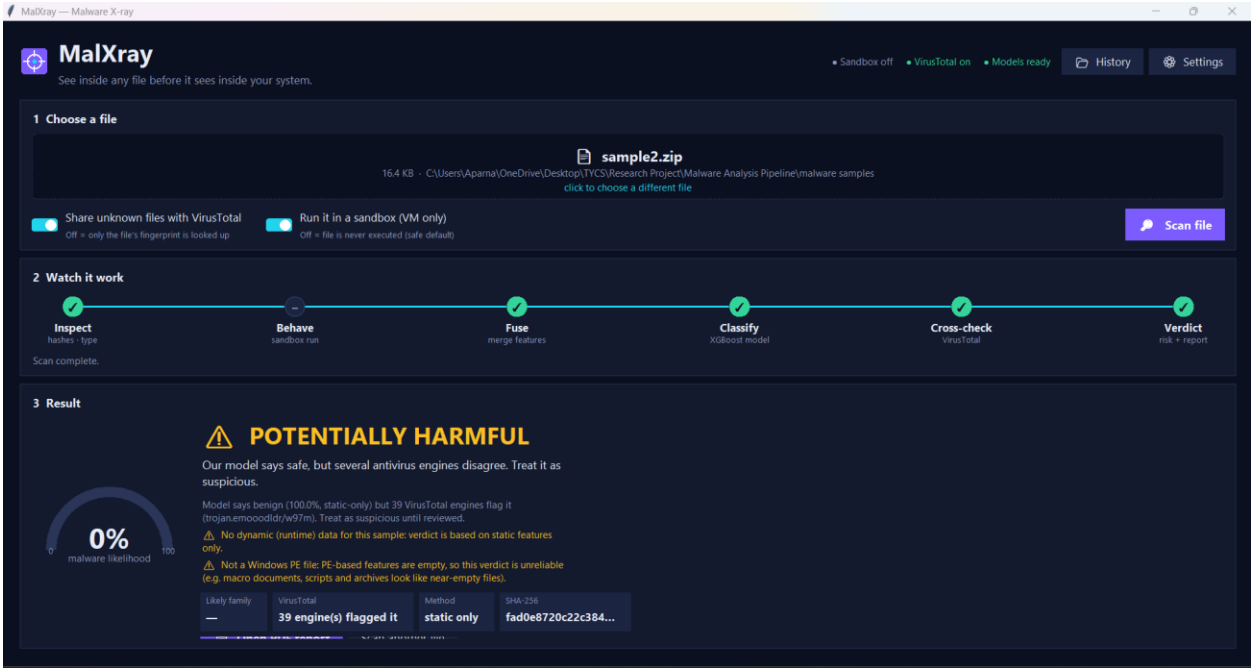
➔ MalXray Dashboard



➔ Choosing the sample file



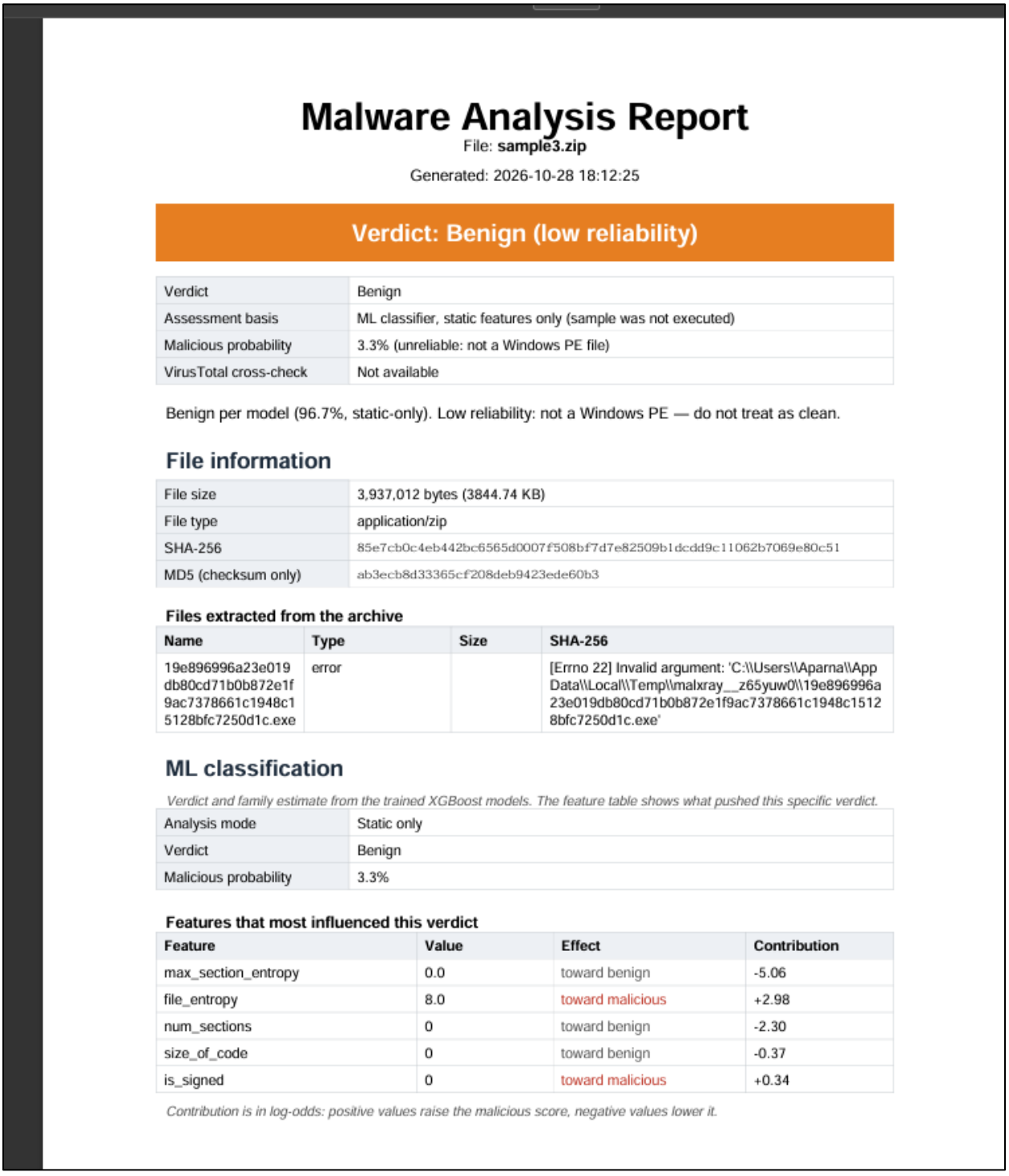
➔ After analysis, the results



The result visualisation presents confidence scores, behavioural summaries and feature importance so that analysts can understand a classification decision and prioritise incident response.

4.5 Report Design

The PDF report is organised into file information, ML verdict and classification mode, confidence and family, behavioural observations, VirusTotal information, feature contributions, and warnings and limitations.



Malware Analysis Report

File: **sample3.zip**

Generated: 2026-10-28 18:12:25

Verdict: Benign (low reliability)

Verdict	Benign
Assessment basis	ML classifier, static features only (sample was not executed)
Malicious probability	3.3% (unreliable: not a Windows PE file)
VirusTotal cross-check	Not available

Benign per model (96.7%, static-only). Low reliability: not a Windows PE — do not treat as clean.

File information

File size	3,937,012 bytes (3844.74 KB)
File type	application/zip
SHA-256	85e7cb0c4eb442bc6565d0007f508bf7d7e82509b1dcdd9c11062b7069e80c51
MD5 (checksum only)	ab3ecb8d33365cf208deb9423ede60b3

Files extracted from the archive

Name	Type	Size	SHA-256
19e896996a23e019db80cd71b0b872e1f9ac7378661c1948c15128bfc7250d1c.exe	error		[Errno 22] Invalid argument: 'C:\\Users\\Aparna\\AppData\\Local\\Temp\\malxray__z65yuw0\\19e896996a23e019db80cd71b0b872e1f9ac7378661c1948c15128bfc7250d1c.exe'

ML classification

Verdict and family estimate from the trained XGBoost models. The feature table shows what pushed this specific verdict.

Analysis mode	Static only
Verdict	Benign
Malicious probability	3.3%

Features that most influenced this verdict

Feature	Value	Effect	Contribution
max_section_entropy	0.0	toward benign	-5.06
file_entropy	8.0	toward malicious	+2.98
num_sections	0	toward benign	-2.30
size_of_code	0	toward benign	-0.37
is_signed	0	toward malicious	+0.34

Contribution is in log-odds: positive values raise the malicious score, negative values lower it.

4.6 Implementation Demonstration

[INSERT screenshots or logs showing: file hashing and type detection; archive handling; static PE feature extraction with section entropy and suspicious APIs; dynamic monitoring output; the 44-feature fused vector; the XGBoost verdict; the VirusTotal cross-check; the generated PDF.]

4.7 Experimental Results [TO BE COMPLETED AFTER FINAL TRAINING]

Report results only after training and testing on an adequate real malware dataset. Suggested tables:

```
[data] Only 0 real sample(s) (< 200 needed to train).
[data] Training on SYNTHETIC data. Metrics below prove the pipeline runs; they are NOT real detection performance.

=== Binary Detector (benign vs malicious) ===
              precision    recall  f1-score   support

      benign       0.99      1.00      1.00       100
   malicious       1.00      1.00      1.00       400

    accuracy                           1.00       500
   macro avg       1.00      1.00      1.00       500
weighted avg       1.00      1.00      1.00       500


=== Family Classifier (malicious samples, known families) ===
              precision    recall  f1-score   support

  ransomware       0.99      0.96      0.97       100
     spyware       1.00      0.97      0.98       100
      trojan       0.95      0.99      0.97       100
        worm       0.97      0.99      0.98       100

    accuracy                           0.98       400
   macro avg       0.98      0.98      0.98       400
weighted avg       0.98      0.98      0.98       400


=== Static vs Dynamic vs Fused Comparison ===
              accuracy  precision  recall      f1  num_features
static_only      0.996     0.9960   0.996  0.9960          37.0
dynamic_only     0.976     0.9786   0.976  0.9765           7.0
fused            0.998     0.9980   0.998  0.9980          44.0

Trained on: SYNTHETIC data. Completed: binary detector, family classifier, static/dynamic/fused comparison.
Models -> 'models/', results -> 'results/'.
```

4.8 Limitations

- Performance depends on dataset quality and diversity.
- Dynamic analysis is limited by how faithfully a sandbox reproduces real conditions and can be evaded by sandbox-aware malware.
- The system performs offline analysis and focuses on Windows PE files.
- There is no real-time detection or kernel-level monitoring.
- Runtime detonation requires an isolated environment and explicit confirmation.

4.9 CONCLUSION AND FUTURE SCOPE

This project presented MalXray, a hybrid malware detection and family classification system that integrates static analysis, dynamic behavioural analysis and machine learning in a unified desktop framework. It addresses key weaknesses of traditional systems in handling obfuscated, zero-day and evolving malware by combining structural and runtime features, using XGBoost with interpretable feature contributions, cross-checking with VirusTotal, and producing a clear PDF report. The modular architecture makes the solution scalable, extensible and suitable for both practical analysis and education. [ADD one sentence summarising your measured results.]

Future scope: real-time detection and continuous monitoring; kernel-level and memory-based analysis; larger and more diverse datasets; richer behavioural monitoring (API-call sequences, registry activity); advanced machine learning and deep learning models; and deployment in practical endpoint-security (EDR) environments.

BIBLIOGRAPHY - REFERENCES AND LINKS

[1] S. Mahajan, R. K. Choudhary, P. Chopde, P. Ravekar, A. Nagre, R. Chitkeshwar and P. Parise, "Next-Gen Malware Detection with Artificial Intelligence," S. B. Jain Institute of Technology, Management & Research, Nagpur, 2024.

[2] A. Damodaran, F. Di Troia, V. A. Corrado, T. H. Austin and M. Stamp, "A Comparison of Static, Dynamic, and Hybrid Analysis for Malware Detection," arXiv preprint, Mar. 2022.

[3] B. Pargi, S. Degadwala and M. Joshi, "Hybrid Malware Analysis using Static and Dynamic Techniques with Machine Learning," Int. J. Sci. Res. Sci. Eng. Technol., vol. 13, no. 1, pp. 22-27, Jan. 2026.

[4] R. S. Leon, M. Kiperberg, A. A. Leon Zabag et al., "Hypervisor-assisted dynamic malware analysis," Cybersecurity, vol. 4, art. 19, Jun. 2021.

[5] H. H. R. Manzil and S. M. Naik, "Android malware category detection using a novel feature vector-based machine learning model," Cybersecurity, vol. 6, art. 6, Mar. 2023.

[6] R. Sharp, "An Introduction to Malware," Technical University of Denmark, Lecture Notes, 2007.

[7] Y.-S. Yen, Z.-W. Chen, Y.-R. Guo and M.-C. Chen, "Integration of Static and Dynamic Analysis for Malware Family Classification with Composite Neural Network," arXiv:1912.11249, Dec. 2019.

[8] U. Bayer, A. Moser, C. Kruegel and E. Kirda, "Dynamic Analysis of Malicious Code," Journal in Computer Virology, vol. 2, no. 1, pp. 67-77, 2006.

[9] M. Egele, T. Scholte, E. Kirda and C. Kruegel, "A Survey on Automated Dynamic Malware Analysis Techniques and Tools," ACM Computing Surveys, vol. 44, no. 2, 2012.

[10] I. Santos, F. Brezo, X. Ugarte-Pedrero and P. G. Bringas, "Opcode Sequence Analysis for Malware Detection," Lecture Notes in Computer Science, vol. 5793, 2009.

[11] A. Mohaisen, O. Alrawi and M. Mohaisen, "AMAL: High-Fidelity, Behavior-Based Automated Malware Analysis and Classification," Computers & Security, vol. 52, pp. 251-266, 2015.

[12] Z. Salehi, M. Sami and M. Ghiasi, "MASTAD: A Hybrid Malware Detection System Based on Static and Dynamic Analysis," J. Information Security and Applications, vol. 45, 2019.

[13] K. Rieck, P. Trinius, C. Willems and T. Holz, "Automatic Analysis of Malware Behavior Using Machine Learning," Journal of Computer Security, vol. 19, no. 4, pp. 639-668, 2011.

[14] D. Ucci, L. Aniello and R. Baldoni, "Survey of Machine Learning Techniques for Malware Analysis," Computers & Security, vol. 81, pp. 123-147, 2019.

[15] H. HaddadPajouh, A. Dehghantanha, R. Khayami and K.-K. R. Choo, "A Deep Learning-Based Approach for Malware Detection Using API Call Sequences," Soft Computing, vol. 23, 2019.

[17] C. Willems, T. Holz and F. Freiling, "Toward Automated Dynamic Malware Analysis Using CWSandbox," IEEE Security & Privacy, vol. 5, no. 2, pp. 32-39, 2007.

[18] J. Z. Kolter and M. A. Maloof, "Learning to Detect and Classify Malicious Executables in the Wild," Journal of Machine Learning Research, vol. 7, pp. 2721-2744, 2006.

[19] L. Nataraj, S. Karthikeyan, G. Jacob and B. S. Manjunath, "Malware Images: Visualization and Automatic Classification," Proc. 8th Int. Symp. Visualization for Cyber Security, 2011.

[20] R. Pascanu, J. W. Stokes, H. Sanossian, M. Marinescu and A. Thomas, "Malware Classification with Recurrent Networks," IEEE ICASSP, 2015.

[22] M. Lindorfer, C. Kolbitsch and P. Milani Comparetti, "Detecting Environment-Sensitive Malware," RAID, 2011.

[23] E. Raff, J. Barker, J. Sylvester, R. Brandon, B. Catanzaro and C. Nicholas, "Malware Detection by Eating a Whole EXE," AAAI Conference on Artificial Intelligence, vol. 32, no. 1, 2018.

Other references from your paper that you can cite where relevant: [16] McLaughlin et al., "Deep Android Malware Detection," CODASPY 2017; [21] Ucci et al. (CyCon 2013 entry, check the author initials and title before citing).

Software and tool links: EMBER dataset (github.com/elastic/ember), Cuckoo Sandbox (cuckoosandbox.org), XGBoost (xgboost.readthedocs.io), pefile (github.com/erocarrera/pefile), LIEF (lief-project.github.io), YARA (virustotal.github.io/yara), VirusTotal (virustotal.com).